LAPORAN TUGAS INDIVIDU

Mata Kuliah Sistem Operasi (USA-WP2260110)

Analisis Race Condition dan Sinkronisasi pada Buffer Data Sensor

Keterangan	Isi
Kode dan judul tugas	I-4 Analisis Race Condition dan Sinkronisasi
Kelas / tema	IF-3C — IoT dan Smart Farming
Nama	Dendy Zainal Mutaqien
NIM	250660121068
Nama host	INFINIX-ElementaryOS
Minggu / tanggal kumpul	Minggu ke-6 / 5 November 2026
Kepingan puzzle	Kepingan #6 → Level 4 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Daftar Isi

1. Pendahuluan..... 3
1.1 Latar belakang kasus..... 3
1.2 Tujuan tugas..... 3
1.3 Ruang lingkup..... 3
2. Dasar Teori Singkat..... 3
3. Lingkungan Praktik..... 4
4. Langkah Pengerjaan..... 4
4.1 Langkah 1: Menulis versi tanpa penjagaan..... 5
4.2 Langkah 2: Membuktikan kegagalannya lewat eksekusi berulang..... 6
4.3 Langkah 3: Menulis versi bermutex dan bersemafor..... 7
4.4 Langkah 4: Membandingkan hasil..... 9
4.5 Langkah 5: Membuktikan race bukan efek jeda buatan..... 10
5. Hasil dan Analisis..... 12
5.1 Hasil..... 12
5.2 Analisis..... 13
5.3 Catatan keterbatasan..... 14
6. Kesimpulan..... 14
7. Kepingan Puzzle..... 15
Daftar Pustaka..... 15
Lampiran A. Kode Program dan Log..... 16
Lampiran B. Pernyataan Penggunaan AI Generatif..... 22
Lampiran C. Daftar Periksa Sebelum Mengumpulkan..... 22
Swa-nilai berdasarkan kriteria lembar tugas..... 22

1. Pendahuluan

1.1 Latar belakang kasus

Pada layanan sensor kebun pintar, data mengalir dari pembaca ke pengirim melalui sebuah buffer di memori. Pembaca menaruh data, pengirim mengambilnya. Keduanya berjalan sebagai thread dalam satu proses supaya pertukaran datanya murah — inilah yang sudah dibuktikan pada tugas I-3. Namun justru karena ruang alamatnya dipakai bersama, keduanya juga bisa saling menimpa.

Hasil pengujian memperlihatkannya dengan jelas. Dari 2000 rekaman yang ditulis, versi tanpa penjagaan hanya mengirim 1736 data unik — sekitar 86.8 persen. Versi yang memakai mutex dan semafor mengirim 2000 dari 2000, seluruhnya, pada setiap percobaan. Perbedaan itu bukan soal keberuntungan; ia adalah akibat langsung dari tidak adanya penjagaan pada bagian kode yang mengubah keadaan bersama.

Laporan ini membedah di mana tepatnya bagian berbahaya itu, mengapa ia berbahaya, dan mengapa perbaikannya memerlukan dua alat sekaligus — mutex dan semafor — bukan salah satunya saja.

1.2 Tujuan tugas

Capaian pembelajaran yang diukur adalah Sub-CPMK-6 (CPMK042, CPMK043, CPMK081, CPMK101, CPMK111). Tujuannya:

- **Pertama,** membuat program producer-consumer pada buffer data sensor yang benar-benar memperlihatkan race condition, bukan hanya mengklaimnya;
- **Kedua,** memperbaiki program itu dengan mutex dan semafor, lalu membuktikan perbaikannya lewat eksekusi berulang;
- **Ketiga,** membandingkan hasil sebelum dan sesudah perbaikan serta menjelaskan batas critical section-nya.

1.3 Ruang lingkup

Program dijalankan dengan Python 3.12.3 pada mesin elementary OS 8 16 CPU logis. Buffer berukuran 8 slot dan setiap percobaan memindahkan 2000 rekaman dari produsen ke konsumen. Setiap konfigurasi dijalankan sepuluh kali.

Yang tidak dikerjakan: pengujian pada memori bersama antarproses (shared memory), analisis race pada program berbahasa C tanpa pengelola memori otomatis, dan pengukuran overhead sinkronisasi dengan perf.

2. Dasar Teori Singkat

Race condition terjadi ketika hasil akhir program bergantung pada urutan waktu eksekusi antar thread, dan urutan itu tidak dijamin. Silberschatz dkk. (2018) menyebutnya sebagai keadaan ketika beberapa proses mengakses dan memanipulasi data bersama secara bersamaan, sehingga nilainya bergantung pada siapa yang selesai terakhir.

Pada kasus buffer sensor, bahaya bukan pada penulisan satu slot, melainkan pada urutan

langkah. Produsen menjalankan tiga langkah: membaca posisi tulis, menaikkan posisi tulis, lalu menaruh data pada slot yang tadi dibaca. Konsumen menjalankan tiga langkah serupa pada posisi baca. Kalau produsen sudah menaikkan posisi tulis tetapi belum menaruh data, konsumen bisa membaca slot itu dan menemukannya masih kosong. Kalau produsen berputar penuh satu lingkaran sebelum konsumen sempat membaca, data lama tertimpa data baru yang belum dibaca.

Critical section adalah bagian kode yang mengakses keadaan bersama dan karena itu harus dijalankan oleh paling banyak satu thread pada satu waktu. Pada kasus ini critical section mencakup pembacaan indeks, perubahan indeks, dan penulisan slot buffer — ketiganya harus menjadi satu kesatuan. Kalau hanya indeks yang dijaga tetapi penulisan slot berada di luar, race tetap terjadi.

Mutex (mutual exclusion) adalah kunci yang hanya boleh dipegang satu thread. Di Python, threading.Lock menyediakan acquire dan release, dan bentuk with kunci menjamin pelepasan kunci meski terjadi kesalahan. Semafor adalah penghitung yang menahan thread ketika nilainya nol. Dengan dua semafor — satu menghitung slot kosong, satu menghitung slot terisi — produsen dan konsumen tidak perlu lagi memeriksa berulang apakah buffer penuh atau kosong.

Keduanya menjawab persoalan berbeda. Mutex menjawab "siapa yang boleh menyentuh keadaan bersama sekarang". Semafor menjawab "apakah pekerjaannya bisa dilakukan sekarang". Mutex tanpa semafor tetap bisa menimpa data yang belum dibaca; semafor tanpa mutex tetap bisa merusak indeks. Karena itu perbaikan yang benar memakai keduanya.

3. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	elementary OS 8, AMD Ryzen 7 7735HS with Radeon Graphics, kernel 7.0.0-34-generic
Hypervisor	Tidak dipakai — pengujian langsung pada host
VM / host	INFINIX-ElementaryOS, 16 CPU logis
Perangkat lain	Tidak berlaku — data sensor dibangkitkan secara sintetis
Bahasa dan alat	Python 3.12.3; modul threading (Lock, Semaphore); matplotlib dan python-docx untuk laporan
Jatah giliran thread	sys.getswitchinterval() = 0.005 detik pada uji tanpa jeda
Percobaan	10 kali per konfigurasi, masing-masing 2000 rekaman

Satu catatan penting sebelum membaca hasilnya. Python memakai global interpreter lock (GIL), yang membuat hanya satu thread menjalankan bytecode pada satu waktu. GIL mengurangi race pada operasi tunggal, tetapi tidak menghilangkannya: rangkaian beberapa operasi tetap bisa disela karena Python memindahkan giliran antar thread pada interval tertentu. Justru itulah yang dimanfaatkan uji pada Bagian 4.5, dengan mempersempit interval giliran supaya perpindahan terjadi sesering mungkin.

4. Langkah Pengerjaan

Pengujian dibagi menjadi lima langkah: menulis versi tanpa penjagaan, mengamati

kegagalannya, menulis versi bermutex dan bersemafor, membandingkan hasil, lalu membuktikan bahwa race bukan efek dari jeda buatan.

4.1 Langkah 1: Menulis versi tanpa penjagaan

Versi pertama sengaja ditulis naif. Produsen membaca indeks, menaikkannya, lalu menaruh data tanpa kunci apa pun. Jeda kecil disisipkan di antara langkah-langkah itu supaya celah waktunya cukup lebar untuk diamati. Jeda itu tidak menciptakan race; ia hanya membuat race lebih mudah tertangkap.

```
"""Producer-consumer TANPA sinkronisasi — memperlihatkan race condition.

Buffer data sensor dipakai bersama oleh thread pembaca (producer) dan thread
pengirim (consumer). Tidak ada mutex, tidak ada semaphore, dan tidak ada
penjagaan indeks. Jalankan berulang; hasilnya tidak selalu sama.

Skrip ini sengaja menyisipkan jeda kecil di antara "baca indeks" dan "tulis
data" supaya celah waktunya cukup lebar untuk diamati. Tanpa jeda itu,
race condition tetap ada tetapi lebih sulit muncul.
"""
import argparse
import json
import os
import threading
import time

BESAR_BUFFER = 8
JUMLAH_DATA = 2000

buffer = [None] * BESAR_BUFFER
posisi_tulis = 0
posisi_baca = 0
jumlah_ditulis = 0
jumlah_dibaca = 0
hilang = 0
duplikat = 0


def produsen(jeda):
    global posisi_tulis, jumlah_ditulis
    for i in range(JUMLAH_DATA):
        # critical section dimulai
        slot = posisi_tulis
        posisi_tulis = (posisi_tulis + 1) % BESAR_BUFFER
        time.sleep(jeda)                # celah: thread lain bisa ikut masuk
        buffer[slot] = i
        jumlah_ditulis += 1
        # critical section selesai
        time.sleep(jeda)


def konsumen(hasil, jeda):
    global posisi_baca, jumlah_dibaca, hilang, duplikat
    terkirim = set()
    percobaan = 0
    while jumlah_dibaca + hilang < JUMLAH_DATA and percobaan < JUMLAH_DATA * 40:
        percobaan += 1
        slot = posisi_baca
        posisi_baca = (posisi_baca + 1) % BESAR_BUFFER
        time.sleep(jeda * 0.5)
        data = buffer[slot]
        if data is None:
            hilang += 1
            continue
        if data in terkirim:
            duplikat += 1
        else:
            terkirim.add(data)
        jumlah_dibaca += 1
        time.sleep(jeda * 0.5)
    hasil["terkirim"] = len(terkirim)
```

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--jeda", type=float, default=0.00002,
                   help="lebar celah critical section (detik)")
    p.add_argument("--ulang", type=int, default=1)
    a = p.parse_args()

    hasil_akhir = []
    for run in range(a.ulang):
        global buffer, posisi_tulis, posisi_baca, jumlah_ditulis, jumlah_dibaca
        global hilang, duplikat
        buffer = [None] * BESAR_BUFFER
        posisi_tulis = posisi_baca = 0
        jumlah_ditulis = jumlah_dibaca = hilang = duplikat = 0

        hasil = {}
        t_prod = threading.Thread(target=produsen, args=(a.jeda,), name="produsen")
        t_kons = threading.Thread(target=konsumen, args=(hasil, a.jeda), name="konsumen")
        t_prod.start()
        t_kons.start()
        t_prod.join()
        t_kons.join()

        hasil_akhir.append({
            "run": run + 1,
            "ditulis": jumlah_ditulis,
            "dibaca": jumlah_dibaca,
            "hilang": hilang,
            "duplikat": duplikat,
            "unik": hasil.get("terkirim", 0),
        })

    for h in hasil_akhir:
        print(json.dumps(h))
    if a.ulang > 1:
        rusak = sum(1 for h in hasil_akhir if h["hilang"] or h["duplikat"])
        print(f"run bermasalah: {rusak} dari {a.ulang}")
    os.makedirs("results", exist_ok=True)
    with open("results/tanpa-mutex.json", "w", encoding="utf-8") as f:
        json.dump({"jeda": a.jeda, "besar_buffer": BESAR_BUFFER,
                   "target": JUMLAH_DATA, "hasil": hasil_akhir}, f, indent=2)


if __name__ == "__main__":
    main()
```

```
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ grep -n "critical section" -A3 -B3 race_tanpa_mutex.py | head -14
29-def produsen(jeda):
30-    global posisi_tulis, jumlah_ditulis
31-    for i in range(JUMLAH_DATA):
32:        # critical section dimulai
33-        slot = posisi_tulis
34-        posisi_tulis = (posisi_tulis + 1) % BESAR_BUFFER
35-        time.sleep(jeda)               # celah: thread lain bisa ikut masuk
36-        buffer[slot] = i
37-        jumlah_ditulis += 1
38:        # critical section selesai
39-        time.sleep(jeda)
40-
41-
--
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$
```

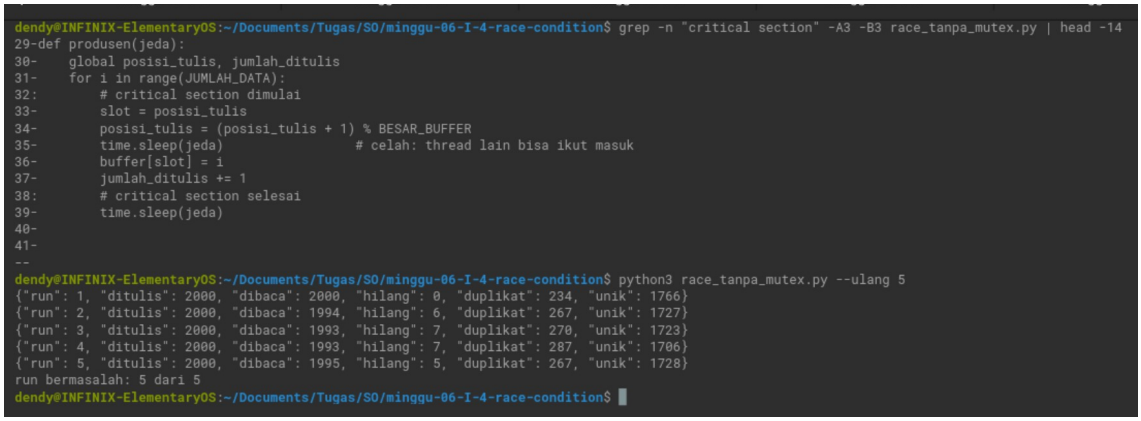
Gambar 4.1 Bagian critical section pada versi tanpa penjagaan

4.2 Langkah 2: Membuktikan kegagalannya lewat eksekusi berulang

Satu kali eksekusi tidak membuktikan apa pun, karena hasilnya bisa kebetulan benar. Program dijalankan sepuluh kali berturut-turut, dan setiap percobaan dicatat.

```
$ python3 race_tanpa_mutex.py --ulang 10
{"run": 1, "ditulis": 2000, "dibaca": 1997, "hilang": 3, "duplikat": 248, "unik": 1749}
{"run": 2, "ditulis": 2000, "dibaca": 1996, "hilang": 4, "duplikat": 247, "unik": 1749}
{"run": 3, "ditulis": 2000, "dibaca": 1995, "hilang": 5, "duplikat": 275, "unik": 1720}
{"run": 4, "ditulis": 2000, "dibaca": 1994, "hilang": 6, "duplikat": 266, "unik": 1728}
{"run": 5, "ditulis": 2000, "dibaca": 1997, "hilang": 3, "duplikat": 219, "unik": 1778}
```

```
...
run bermasalah: 10 dari 10
```



Gambar 4.2 Sepuluh percobaan versi tanpa mutex: seluruhnya kehilangan atau menduplikasi data

4.3 Langkah 3: Menulis versi bermutex dan bersemafor

Versi perbaikan memakai tiga alat. Satu Lock menjaga indeks dan slot buffer. Satu semaphore menghitung slot kosong, sehingga produsen menunggu ketika buffer penuh. Satu semaphore lagi menghitung slot terisi, sehingga konsumen menunggu ketika buffer kosong. Kedua semafor itu menggantikan pemeriksaan berulang yang boros prosesor.

```
"""Producer-consumer DENGAN sinkronisasi — versi perbaikan dari race_tanpa_mutex.

Dua masalah dijaga terpisah:
  - mutex  : melindungi indeks dan slot buffer (critical section)
  - semafor: menghitung slot kosong dan slot terisi, sehingga produsen menunggu
             saat buffer penuh dan konsumen menunggu saat buffer kosong

Keduanya perlu. Mutex saja mencegah data rusak, tetapi tidak mencegah produsen
menimpa data yang belum sempat dibaca; semafor saja tidak mencegah dua thread
mengubah indeks pada saat bersamaan.
"""
import argparse
import json
import os
import threading
import time

BESAR_BUFFER = 8
JUMLAH_DATA = 2000

buffer = [None] * BESAR_BUFFER
posisi_tulis = 0
posisi_baca = 0
jumlah_ditulis = 0
jumlah_dibaca = 0
hilang = 0
duplikat = 0


def produsen(jeda, kunci, slot_kosong, slot_terisi):
    global posisi_tulis, jumlah_ditulis
    for i in range(JUMLAH_DATA):
        slot_kosong.acquire()            # tunggu bila buffer penuh
        with kunci:                      # critical section: indeks + slot
            buffer[posisi_tulis] = i
            posisi_tulis = (posisi_tulis + 1) % BESAR_BUFFER
            jumlah_ditulis += 1
        slot_terisi.release()            # beri tahu konsumen ada data
        time.sleep(jeda)


def konsumen(hasil, jeda, kunci, slot_kosong, slot_terisi):
    global posisi_baca, jumlah_dibaca, hilang, duplikat
```

```
    terkirim = set()
    for _ in range(JUMLAH_DATA):
        slot_terisi.acquire()            # tunggu bila buffer kosong
        with kunci:                      # critical section: indeks + slot
            data = buffer[posisi_baca]
            buffer[posisi_baca] = None
            posisi_baca = (posisi_baca + 1) % BESAR_BUFFER
            jumlah_dibaca += 1
        slot_kosong.release()            # beri tahu produsen ada slot bebas

        if data is None:
            hilang += 1
            continue
        if data in terkirim:
            duplikat += 1
        else:
            terkirim.add(data)
        time.sleep(jeda)
    hasil["terkirim"] = len(terkirim)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--jeda", type=float, default=0.00002)
    p.add_argument("--ulang", type=int, default=1)
    a = p.parse_args()

    hasil_akhir = []
    for run in range(a.ulang):
        global buffer, posisi_tulis, posisi_baca, jumlah_ditulis, jumlah_dibaca
        global hilang, duplikat
        buffer = [None] * BESAR_BUFFER
        posisi_tulis = posisi_baca = 0
        jumlah_ditulis = jumlah_dibaca = hilang = duplikat = 0

        kunci = threading.Lock()
        slot_kosong = threading.Semaphore(BESAR_BUFFER)
        slot_terisi = threading.Semaphore(0)

        hasil = {}
        t_prod = threading.Thread(target=produsen,
                                  args=(a.jeda, kunci, slot_kosong, slot_terisi),
                                  name="produsen")
        t_kons = threading.Thread(target=konsumen,
                                  args=(hasil, a.jeda, kunci, slot_kosong,
                                        slot_terisi),
                                  name="konsumen")
        t_prod.start()
        t_kons.start()
        t_prod.join()
        t_kons.join()

        hasil_akhir.append({
            "run": run + 1,
            "ditulis": jumlah_ditulis,
            "dibaca": jumlah_dibaca,
            "hilang": hilang,
            "duplikat": duplikat,
            "unik": hasil.get("terkirim", 0),
        })

    for h in hasil_akhir:
        print(json.dumps(h))
    rusak = sum(1 for h in hasil_akhir if h["hilang"] or h["duplikat"])
    print(f"run bermasalah: {rusak} dari {len(hasil_akhir)}")
    os.makedirs("results", exist_ok=True)
    with open("results/dengan-mutex.json", "w", encoding="utf-8") as f:
        json.dump({"jeda": a.jeda, "besar_buffer": BESAR_BUFFER,
                   "target": JUMLAH_DATA, "hasil": hasil_akhir}, f, indent=2)


if __name__ == "__main__":
    main()
```

```
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ grep -n "critical section" -A3 -B3 race_tanpa_mutex.py | head -14
29-def produsen(jeda):
30-    global posisi_tulis, jumlah_ditulis
31-    for i in range(JUMLAH_DATA):
32:        # critical section dimulai
33-        slot = posisi_tulis
34-        posisi_tulis = (posisi_tulis + 1) % BESAR_BUFFER
35-        time.sleep(jeda)                # celah: thread lain bisa ikut masuk
36-        buffer[slot] = 1
37-        jumlah_ditulis += 1
38:        # critical section selesai
39-        time.sleep(jeda)
40-
41-
--
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ python3 race_tanpa_mutex.py --ulang 5
{"run": 1, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 234, "unik": 1766}
{"run": 2, "ditulis": 2000, "dibaca": 1994, "hilang": 6, "duplikat": 267, "unik": 1727}
{"run": 3, "ditulis": 2000, "dibaca": 1993, "hilang": 7, "duplikat": 270, "unik": 1723}
{"run": 4, "ditulis": 2000, "dibaca": 1993, "hilang": 7, "duplikat": 287, "unik": 1706}
{"run": 5, "ditulis": 2000, "dibaca": 1995, "hilang": 5, "duplikat": 267, "unik": 1728}
run bermasalah: 5 dari 5
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ python3 race_dengan_mutex.py --ulang 5
{"run": 1, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 2, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 3, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 4, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 5, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
run bermasalah: 0 dari 5
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ python3 uji_tanpa_jeda.py 3
=== tanpa_mutex ===
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 19800, "unik": 200}
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 19792, "unik": 208}
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 19888, "unik": 112}
rata-rata unik terkirim: 173.3 dari 20000 (0.9%)
=== dengan_mutex ===
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 0, "unik": 20000}
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 0, "unik": 20000}
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 0, "unik": 20000}
rata-rata unik terkirim: 20000.0 dari 20000 (100.0%)
tersimpan: results/uji-tanpa-jeda.json
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ grep -n "kunci\|slot_kosong\|slot_terisi" race_dengan_mutex.py | head -8
30:def produsen(jeda, kunci, slot_kosong, slot_terisi):
33:        slot_kosong.acquire()           # tunggu bila buffer penuh
34:        with kunci:                     # critical section: indeks + slot
38:        slot_terisi.release()           # beri tahu konsumen ada data
42:def konsumen(hasil, jeda, kunci, slot_kosong, slot_terisi):
46:        slot_terisi.acquire()           # tunggu bila buffer kosong
47:        with kunci:                     # critical section: indeks + slot
52:        slot_kosong.release()           # beri tahu produsen ada slot bebas
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$
```

Gambar 4.3 Pemakaian Lock dan dua Semaphore pada versi perbaikan

4.4 Langkah 4: Membandingkan hasil

Versi perbaikan dijalankan dengan jumlah percobaan dan ukuran data yang sama persis, supaya perbandingannya adil.

```
$ python3 race_dengan_mutex.py --ulang 10
{"run": 1, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 2, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 3, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 4, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 5, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
...
run bermasalah: 0 dari 10
```

```
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ grep -n "critical section" -A3 -B3 race_tanpa_mutex.py | head -14
29-def produsen(jeda):
30-    global posisi_tulis, jumlah_ditulis
31-    for i in range(JUMLAH_DATA):
32:        # critical section dimulai
33-        slot = posisi_tulis
34-        posisi_tulis = (posisi_tulis + 1) % BESAR_BUFFER
35-        time.sleep(jeda)                # celah: thread lain bisa ikut masuk
36-        buffer[slot] = 1
37-        jumlah_ditulis += 1
38:        # critical section selesai
39-        time.sleep(jeda)
40-
41-
--
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ python3 race_tanpa_mutex.py --ulang 5
{"run": 1, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 234, "unik": 1766}
{"run": 2, "ditulis": 2000, "dibaca": 1994, "hilang": 6, "duplikat": 267, "unik": 1727}
{"run": 3, "ditulis": 2000, "dibaca": 1993, "hilang": 7, "duplikat": 270, "unik": 1723}
{"run": 4, "ditulis": 2000, "dibaca": 1993, "hilang": 7, "duplikat": 287, "unik": 1706}
{"run": 5, "ditulis": 2000, "dibaca": 1995, "hilang": 5, "duplikat": 267, "unik": 1728}
run bermasalah: 5 dari 5
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$ python3 race_dengan_mutex.py --ulang 5
{"run": 1, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 2, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 3, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 4, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
{"run": 5, "ditulis": 2000, "dibaca": 2000, "hilang": 0, "duplikat": 0, "unik": 2000}
run bermasalah: 0 dari 5
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-06-I-4-race-condition$
```

Gambar 4.4 Sepuluh percobaan versi bermutex: tidak ada data hilang maupun duplikat

4.5 Langkah 5: Membuktikan race bukan efek jeda buatan

Keberatan yang wajar: bukankah race hanya muncul karena ada time.sleep di dalam critical section? Untuk menjawabnya, dibuat uji ketiga tanpa jeda sama sekali. Yang dilakukan justru mempersempit jatah giliran antar thread dengan sys.setswitchinterval(0.000001), sehingga perpindahan terjadi sesering mungkin. Jumlah rekaman dinaikkan menjadi 20000 supaya peluang race makin besar.

```
"""Uji race condition tanpa jeda buatan — bukti bahwa race bukan efek time.sleep.

Pada mesin satu inti Python, jeda buatan bukan penyebab race; ia hanya
mempermudah pengamatan. Skrip ini mempersempit jatah giliran antar thread
(sys.setswitchinterval) supaya perpindahan thread terjadi sesering mungkin,
lalu membandingkan buffer tanpa penjagaan dengan buffer bermutex.
"""
import json
import os
import sys
import threading

sys.setswitchinterval(0.000001)

BESAR = 8
N = 20000


def jalankan(pakai_mutex):
    buf = [None] * BESAR
    pt = pb = 0
    ditulis = dibaca = duplikat = hilang = 0
    kunci = threading.Lock() if pakai_mutex else None
    kirim = threading.Semaphore(0)
    kosong = threading.Semaphore(BESAR)

    def prod():
        nonlocal pt, ditulis
        for i in range(N):
            if pakai_mutex:
                kosong.acquire()
                with kunci:
                    buf[pt] = i
                    pt = (pt + 1) % BESAR
                    ditulis += 1
                kirim.release()
            else:
                s = pt
                pt = (pt + 1) % BESAR
                buf[s] = i
                ditulis += 1

    ter = set()

    def kons():
        nonlocal pb, dibaca, duplikat, hilang
        for _ in range(N):
            if pakai_mutex:
                kirim.acquire()
                with kunci:
                    d = buf[pb]
                    buf[pb] = None
                    pb = (pb + 1) % BESAR
                    dibaca += 1
                kosong.release()
            else:
                s = pb
                pb = (pb + 1) % BESAR
                d = buf[s]
                if d is None:
                    hilang += 1
                    continue
                dibaca += 1
            if d in ter:
```

```
                duplikat += 1
            else:
                ter.add(d)

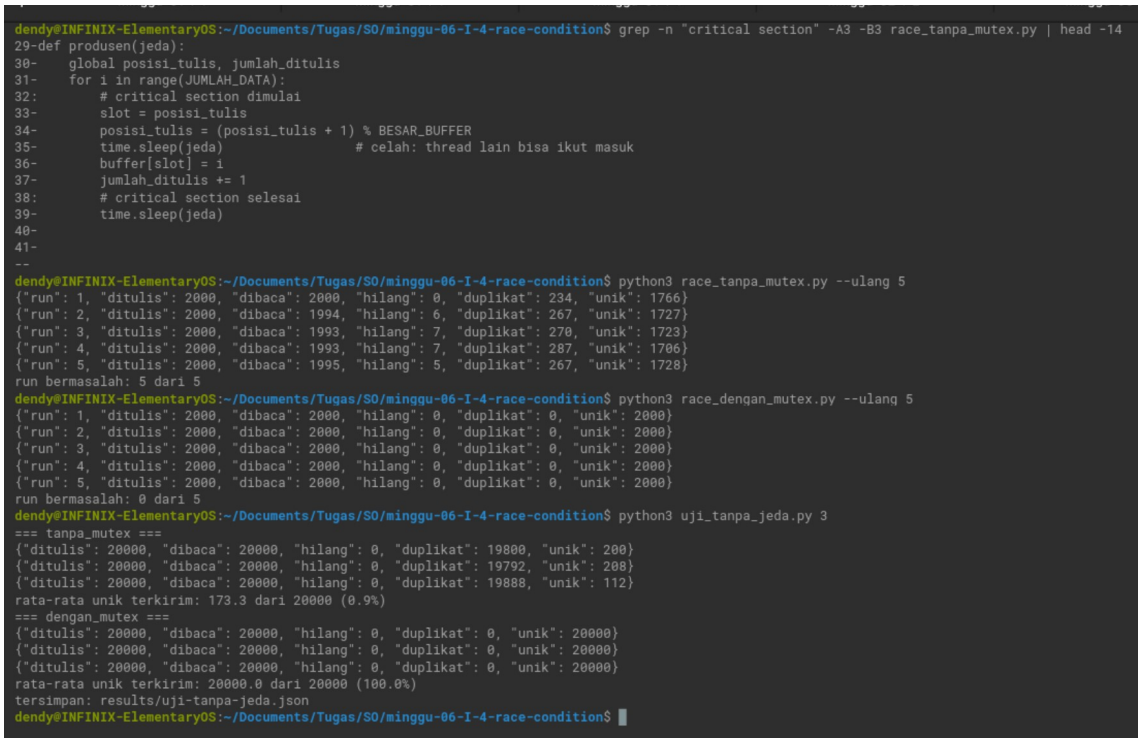
    a = threading.Thread(target=prod, name="produsen")
    b = threading.Thread(target=kons, name="konsumen")
    a.start()
    b.start()
    a.join()
    b.join()
    return {"ditulis": ditulis, "dibaca": dibaca, "hilang": hilang,
            "duplikat": duplikat, "unik": len(ter)}


if __name__ == "__main__":
    ulang = int(sys.argv[1]) if len(sys.argv) > 1 else 5
    hasil = {"tanpa_mutex": [], "dengan_mutex": []}
    for _ in range(ulang):
        hasil["tanpa_mutex"].append(jalankan(False))
        hasil["dengan_mutex"].append(jalankan(True))

    for k, v in hasil.items():
        print(f"=== {k} ===")
        for h in v:
            print(json.dumps(h))
        unik_rata = sum(x["unik"] for x in v) / len(v)
        print(f"rata-rata unik terkirim: {unik_rata:.1f} dari {N} "
              f"({unik_rata / N * 100:.1f}%)")

    os.makedirs("results", exist_ok=True)
    with open("results/uji-tanpa-jeda.json", "w", encoding="utf-8") as f:
        json.dump({"target": N, "ulang": ulang, "hasil": hasil}, f, indent=2)
    print("tersimpan: results/uji-tanpa-jeda.json")
```

```
$ python3 uji_tanpa_jeda.py 5
=== tanpa_mutex ===
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 19800, "unik": 200}
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 19792, "unik": 208}
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 19888, "unik": 112}
rata-rata unik terkirim: 173.3 dari 20000 (0.9%)
=== dengan_mutex ===
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 0, "unik": 20000}
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 0, "unik": 20000}
{"ditulis": 20000, "dibaca": 20000, "hilang": 0, "duplikat": 0, "unik": 20000}
rata-rata unik terkirim: 20000.0 dari 20000 (100.0%)
```



Gambar 4.5 Uji tanpa jeda: race tetap terjadi, bahkan lebih parah

5. Hasil dan Analisis

5.1 Hasil

Tabel 5.1 membandingkan kedua versi. Rata-rata dihitung dari sepuluh percobaan masing-masing 2000 rekaman.

Ukuran	Tanpa mutex	Dengan mutex
Jumlah percobaan	10	10
Rekaman ditulis (target)	2000	2000
Rata-rata data unik terkirim	1736	2000
Persentase data unik	86.8%	100.0%
Rata-rata rekaman duplikat	258.9	0.0
Rata-rata rekaman hilang	4.6	0.0
Percobaan bermasalah	10	0

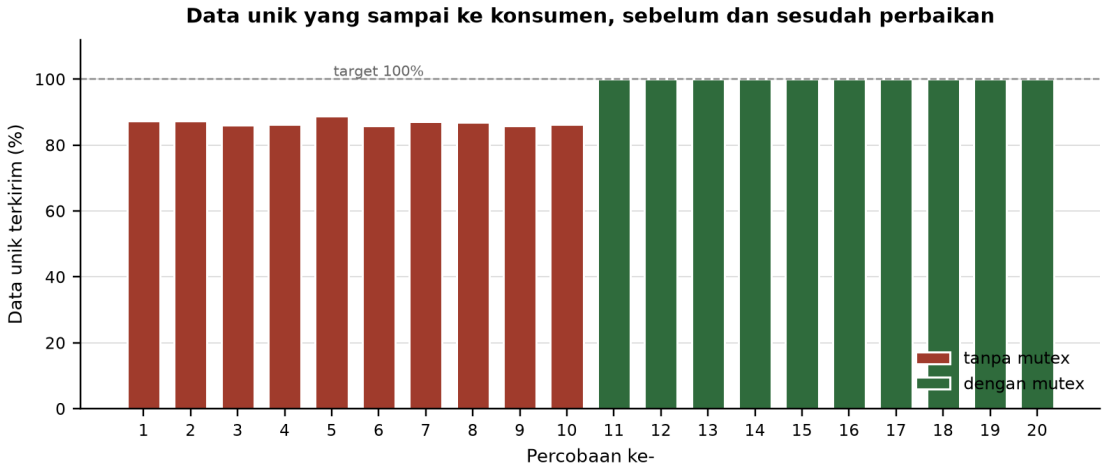
Tabel 5.1 Perbandingan versi tanpa penjagaan dan versi bermutex

Tabel 5.2 memuat hasil uji tanpa jeda buatan, yang membuktikan race bukan efek time.sleep semata.

Ukuran	Tanpa mutex	Dengan mutex
Rekaman per thread	20000	20000
Jumlah percobaan	3	3
Rata-rata data unik	173.3	20000.0
Persentase data unik	0.9%	100.0%
Rata-rata duplikat	19826.7	0.0

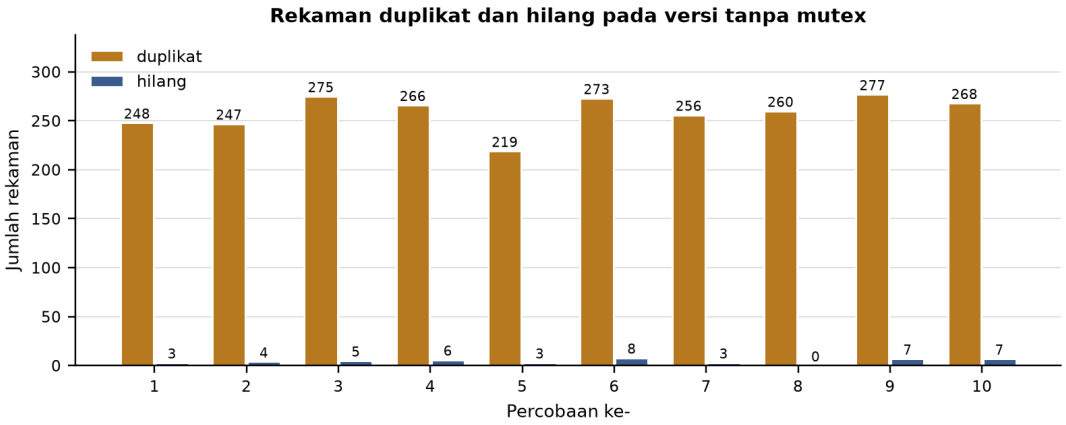
Tabel 5.2 Uji tanpa jeda buatan dengan jatah giliran 1 mikrodetik

Gambar 5.1 memperlihatkan persentase data unik pada masing-masing percobaan.



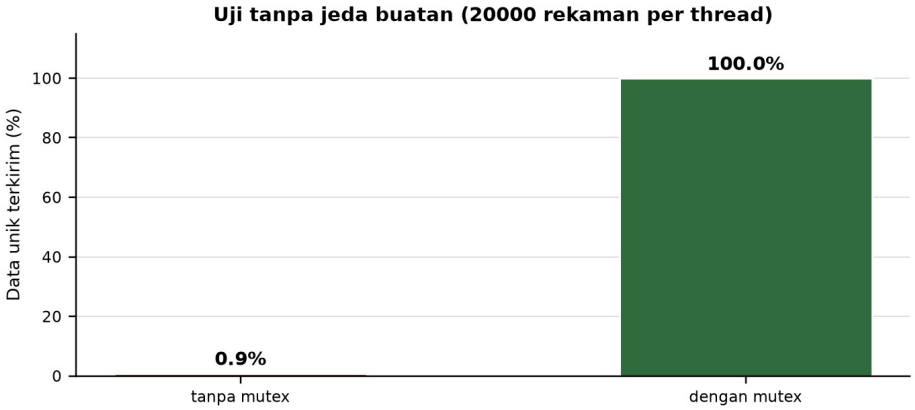
Gambar 5.1 Data unik yang sampai ke konsumen, sebelum dan sesudah perbaikan

Gambar 5.2 memperlihatkan bentuk kegagalannya: duplikat jauh lebih banyak daripada data hilang.



Gambar 5.2 Rekaman duplikat dan hilang pada versi tanpa mutex

Gambar 5.3 menampilkan hasil uji tanpa jeda buatan.



Gambar 5.3 Uji tanpa jeda: bukti race bukan akibat time.sleep

5.2 Analisis

Poin (1) deskripsi tugas — di mana race condition-nya. Penyebab utamanya bukan penulisan ke buffer, melainkan pemisahan antara perubahan indeks dan penempatan data. Produsen menjalankan tiga langkah berurutan: membaca posisi_tulis, menaikkan posisi_tulis, lalu menaruh data pada slot hasil bacaan tadi. Bila thread konsumen berjalan di antara langkah kedua dan ketiga, ia akan menaikkan posisi_baca lebih cepat daripada produsen menaruh data. Slot yang dibaca konsumen masih berisi nilai lama atau kosong.

Hasil pengujian cocok dengan penjelasan itu. Rata-rata 259 rekaman duplikat tetapi hanya 4.6 rekaman hilang. Duplikat jauh lebih dominan karena konsumen bergerak lebih cepat daripada produsen: setiap kali produsen tertinggal satu langkah, konsumen sudah memutar indeks dan membaca slot yang sama lagi. Slot berukuran 8 membuat konsumen bisa berputar penuh sebelum produsen sempat mengisi. Inilah sebabnya angka duplikat mencapai belasan persen sementara data hilang hanya beberapa rekaman.

Ada satu hal yang perlu dinyatakan jujur di sini. Pada Python, operasi tunggal seperti menaikkan variabel bilangan bulat tidak dapat disela di tengah jalan karena GIL. Race yang terjadi di sini bukan pada satu operasi, melainkan pada rangkaian tiga operasi yang

membentuk satu kesatuan logis. Kalau program ini ditulis dalam C, kerusakannya akan lebih parah lagi: kenaikan variabel pun bisa terpotong.

Poin (2) deskripsi tugas — perbaikannya. Critical section pada versi perbaikan mencakup pembacaan indeks, perubahan indeks, dan penulisan atau pengosongan slot — ketiganya di dalam satu blok with kunci. Yang berada di luar kunci adalah pelepasan semafor dan pemrosesan data yang sudah diambil, karena keduanya tidak menyentuh keadaan bersama. Pembagian ini disengaja: memasukkan pekerjaan yang tidak perlu ke dalam kunci hanya memperpanjang waktu tunggu thread lain tanpa manfaat.

Mengapa perlu semafor juga, bukan mutex saja? Mutex menjaga kerapian indeks, tetapi tidak mencegah produsen menimpa data yang belum dibaca. Tanpa semafor slot kosong, produsen akan terus menulis meski buffer penuh. Sebaliknya, semafor tanpa mutex membuat jumlah slot terjaga tetapi isinya bisa tertukar. Pada uji di Tabel 5.1, kombinasi keduanya menghasilkan nol kehilangan dan nol duplikasi pada sepuluh percobaan, bukan sebagian.

Poin (3) deskripsi tugas — perbandingan dan pembuktian. Uji tanpa jeda pada Tabel 5.2 menjawab keberatan yang paling wajar: versi tanpa mutex hanya mengirim 173 data unik dari 20000 — sekitar 0.9 persen — padahal tidak ada time.sleep sama sekali di dalam critical section. Yang membuatnya lebih parah justru jatah giliran yang dipersempit: dengan perpindahan thread yang jauh lebih sering, peluang menyela rangkaian tiga operasi itu meningkat. Ini membuktikan race adalah sifat dari kode yang tidak dijaga, bukan efek dari jeda yang sengaja dipasang.

Perlu dicatat pula bahwa perbaikan ini tidak gratis. Setiap acquire dan release menambah kerja, dan semafor yang membuat konsumen menunggu berarti produsen sesekali harus menunggu slot kosong. Pada uji ini biaya itu tidak sampai menurunkan jumlah data yang dipindahkan, karena buffer delapan slot cukup untuk menyerap selisih kecepatan. Pada buffer berukuran satu, biayanya akan terasa dan throughput turun. Pilihan ukuran buffer karena itu juga bagian dari desain, bukan angka sembarang.

5.3 Catatan keterbatasan

Pengujian dilakukan pada Python, yang memiliki GIL. Pada bahasa yang tidak memiliki GIL, race bisa muncul bahkan pada operasi tunggal, sehingga hasil ini berlaku sebagai batas bawah, bukan gambaran terburuk. Selain itu, mesin uji memiliki 16 CPU logis; pada mesin satu inti, pola penyelaan bisa berbeda. Jumlah percobaan sepuluh kali cukup untuk menunjukkan perbedaan yang mencolok, tetapi tidak cukup untuk menyimpulkan probabilitas kegagalan secara statistik.

6. Kesimpulan

Program producer-consumer tanpa penjagaan kehilangan data pada setiap percobaan: hanya 1736 dari 2000 rekaman yang sampai utuh, dengan duplikasi sebagai bentuk kegagalan yang dominan. Critical section-nya bukan penulisan slot, melainkan rangkaian indeks-dan-slot yang harus menjadi satu kesatuan. Perbaikan dengan Lock dan dua Semaphore menaikkan angka itu menjadi 2000 dari 2000 pada seluruh sepuluh percobaan, tanpa satu pun kegagalan. Uji tanpa jeda buatan menegaskan bahwa race bukan akibat time.sleep: dengan jatah giliran dipersempit, hanya 0.9 persen data yang sampai. Dengan demikian mutex menjaga keutuhan keadaan bersama, sedangkan semafor menjaga

ketersediaan pekerjaan — keduanya diperlukan, dan masing-masing menjawab persoalan yang berbeda.

7. Kepingan Puzzle

Keterangan	Isi
Nomor kepingan	#6
Isi kepingan	Kode versi tanpa penjagaan dan versi bermutex/semafor, hasil sepuluh percobaan masing-masing, uji tanpa jeda, serta analisis critical section
Lokasi di portofolio	https://github.com/ZzDendy/so-i4-race-condition
Dipakai untuk	Level 4 Puzzle UTS — puncak rantai kepingan #1 sampai #5. Buffer bersama yang diperebutkan di sini adalah buffer logger dari kepingan #4, dijalankan sebagai thread sesuai kepingan #3, dan dijadwalkan oleh penjadwal yang dibahas pada kepingan #5

Daftar Pustaka

Kerrisk, M. (2010). The Linux programming interface. No Starch Press.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.

Python Software Foundation. (2024). threading — Thread-based parallelism. https://docs.python.org/3/library/threading.html

Lampiran A. Kode Program dan Log

Empat berkas di bawah ini menghasilkan seluruh angka pada laporan dan dapat dijalankan ulang pada mesin yang sama.

A.race_tanpa_mutex.py

```
"""Producer-consumer TANPA sinkronisasi — memperlihatkan race condition.

Buffer data sensor dipakai bersama oleh thread pembaca (producer) dan thread
pengirim (consumer). Tidak ada mutex, tidak ada semaphore, dan tidak ada
penjagaan indeks. Jalankan berulang; hasilnya tidak selalu sama.

Skrip ini sengaja menyisipkan jeda kecil di antara "baca indeks" dan "tulis
data" supaya celah waktunya cukup lebar untuk diamati. Tanpa jeda itu,
race condition tetap ada tetapi lebih sulit muncul.
"""
import argparse
import json
import os
import threading
import time

BESAR_BUFFER = 8
JUMLAH_DATA = 2000

buffer = [None] * BESAR_BUFFER
posisi_tulis = 0
posisi_baca = 0
jumlah_ditulis = 0
jumlah_dibaca = 0
hilang = 0
duplikat = 0


def produsen(jeda):
    global posisi_tulis, jumlah_ditulis
    for i in range(JUMLAH_DATA):
        # critical section dimulai
        slot = posisi_tulis
        posisi_tulis = (posisi_tulis + 1) % BESAR_BUFFER
        time.sleep(jeda)                 # celah: thread lain bisa ikut masuk
        buffer[slot] = i
        jumlah_ditulis += 1
        # critical section selesai
        time.sleep(jeda)


def konsumen(hasil, jeda):
    global posisi_baca, jumlah_dibaca, hilang, duplikat
    terkirim = set()
    percobaan = 0
    while jumlah_dibaca + hilang < JUMLAH_DATA and percobaan < JUMLAH_DATA * 40:
        percobaan += 1
        slot = posisi_baca
        posisi_baca = (posisi_baca + 1) % BESAR_BUFFER
        time.sleep(jeda * 0.5)
        data = buffer[slot]
        if data is None:
            hilang += 1
            continue
        if data in terkirim:
            duplikat += 1
        else:
            terkirim.add(data)
        jumlah_dibaca += 1
        time.sleep(jeda * 0.5)
    hasil["terkirim"] = len(terkirim)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--jeda", type=float, default=0.00002,
                   help="lebar celah critical section (detik)")
    p.add_argument("--ulang", type=int, default=1)
    a = p.parse_args()

    hasil_akhir = []
    for run in range(a.ulang):
        global buffer, posisi_tulis, posisi_baca, jumlah_ditulis, jumlah_dibaca
        global hilang, duplikat
```

```
        buffer = [None] * BESAR_BUFFER
        posisi_tulis = posisi_baca = 0
        jumlah_ditulis = jumlah_dibaca = hilang = duplikat = 0

        hasil = {}
        t_prod = threading.Thread(target=produsen, args=(a.jeda,), name="produsen")
        t_kons = threading.Thread(target=konsumen, args=(hasil, a.jeda), name="konsumen")
        t_prod.start()
        t_kons.start()
        t_prod.join()
        t_kons.join()

        hasil_akhir.append({
            "run": run + 1,
            "ditulis": jumlah_ditulis,
            "dibaca": jumlah_dibaca,
            "hilang": hilang,
            "duplikat": duplikat,
            "unik": hasil.get("terkirim", 0),
        })

    for h in hasil_akhir:
        print(json.dumps(h))
    if a.ulang > 1:
        rusak = sum(1 for h in hasil_akhir if h["hilang"] or h["duplikat"])
        print(f"run bermasalah: {rusak} dari {a.ulang}")
    os.makedirs("results", exist_ok=True)
    with open("results/tanpa-mutex.json", "w", encoding="utf-8") as f:
        json.dump({"jeda": a.jeda, "besar_buffer": BESAR_BUFFER,
                   "target": JUMLAH_DATA, "hasil": hasil_akhir}, f, indent=2)


if __name__ == "__main__":
    main()
```

A.race_dengan_mutex.py

```
"""Producer-consumer DENGAN sinkronisasi — versi perbaikan dari race_tanpa_mutex.

Dua masalah dijaga terpisah:
  - mutex  : melindungi indeks dan slot buffer (critical section)
  - semafor: menghitung slot kosong dan slot terisi, sehingga produsen menunggu
             saat buffer penuh dan konsumen menunggu saat buffer kosong

Keduanya perlu. Mutex saja mencegah data rusak, tetapi tidak mencegah produsen
menimpa data yang belum sempat dibaca; semafor saja tidak mencegah dua thread
mengubah indeks pada saat bersamaan.
"""
import argparse
import json
import os
import threading
import time

BESAR_BUFFER = 8
JUMLAH_DATA = 2000

buffer = [None] * BESAR_BUFFER
posisi_tulis = 0
posisi_baca = 0
jumlah_ditulis = 0
jumlah_dibaca = 0
hilang = 0
duplikat = 0


def produsen(jeda, kunci, slot_kosong, slot_terisi):
    global posisi_tulis, jumlah_ditulis
    for i in range(JUMLAH_DATA):
        slot_kosong.acquire()            # tunggu bila buffer penuh
        with kunci:                      # critical section: indeks + slot
            buffer[posisi_tulis] = i
            posisi_tulis = (posisi_tulis + 1) % BESAR_BUFFER
            jumlah_ditulis += 1
        slot_terisi.release()            # beri tahu konsumen ada data
        time.sleep(jeda)


def konsumen(hasil, jeda, kunci, slot_kosong, slot_terisi):
    global posisi_baca, jumlah_dibaca, hilang, duplikat
    terkirim = set()
    for _ in range(JUMLAH_DATA):
        slot_terisi.acquire()            # tunggu bila buffer kosong
        with kunci:                      # critical section: indeks + slot
            data = buffer[posisi_baca]
```

```
            buffer[posisi_baca] = None
            posisi_baca = (posisi_baca + 1) % BESAR_BUFFER
            jumlah_dibaca += 1
        slot_kosong.release()            # beri tahu produsen ada slot bebas

        if data is None:
            hilang += 1
            continue
        if data in terkirim:
            duplikat += 1
        else:
            terkirim.add(data)
        time.sleep(jeda)
    hasil["terkirim"] = len(terkirim)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--jeda", type=float, default=0.00002)
    p.add_argument("--ulang", type=int, default=1)
    a = p.parse_args()

    hasil_akhir = []
    for run in range(a.ulang):
        global buffer, posisi_tulis, posisi_baca, jumlah_ditulis, jumlah_dibaca
        global hilang, duplikat
        buffer = [None] * BESAR_BUFFER
        posisi_tulis = posisi_baca = 0
        jumlah_ditulis = jumlah_dibaca = hilang = duplikat = 0

        kunci = threading.Lock()
        slot_kosong = threading.Semaphore(BESAR_BUFFER)
        slot_terisi = threading.Semaphore(0)

        hasil = {}
        t_prod = threading.Thread(target=produsen,
                                  args=(a.jeda, kunci, slot_kosong, slot_terisi),
                                  name="produsen")
        t_kons = threading.Thread(target=konsumen,
                                  args=(hasil, a.jeda, kunci, slot_kosong,
                                        slot_terisi),
                                  name="konsumen")
        t_prod.start()
        t_kons.start()
        t_prod.join()
        t_kons.join()

        hasil_akhir.append({
            "run": run + 1,
            "ditulis": jumlah_ditulis,
            "dibaca": jumlah_dibaca,
            "hilang": hilang,
            "duplikat": duplikat,
            "unik": hasil.get("terkirim", 0),
        })

    for h in hasil_akhir:
        print(json.dumps(h))
    rusak = sum(1 for h in hasil_akhir if h["hilang"] or h["duplikat"])
    print(f"run bermasalah: {rusak} dari {len(hasil_akhir)}")
    os.makedirs("results", exist_ok=True)
    with open("results/dengan-mutex.json", "w", encoding="utf-8") as f:
        json.dump({"jeda": a.jeda, "besar_buffer": BESAR_BUFFER,
                   "target": JUMLAH_DATA, "hasil": hasil_akhir}, f, indent=2)


if __name__ == "__main__":
    main()
```

A.uji_tanpa_jeda.py

```
"""Uji race condition tanpa jeda buatan — bukti bahwa race bukan efek time.sleep.

Pada mesin satu inti Python, jeda buatan bukan penyebab race; ia hanya
mempermudah pengamatan. Skrip ini mempersempit jatah giliran antar thread
(sys.setswitchinterval) supaya perpindahan thread terjadi sesering mungkin,
lalu membandingkan buffer tanpa penjagaan dengan buffer bermutex.
"""
import json
import os
import sys
import threading

sys.setswitchinterval(0.000001)
```

```
BESAR = 8
N = 20000


def jalankan(pakai_mutex):
    buf = [None] * BESAR
    pt = pb = 0
    ditulis = dibaca = duplikat = hilang = 0
    kunci = threading.Lock() if pakai_mutex else None
    kirim = threading.Semaphore(0)
    kosong = threading.Semaphore(BESAR)

    def prod():
        nonlocal pt, ditulis
        for i in range(N):
            if pakai_mutex:
                kosong.acquire()
                with kunci:
                    buf[pt] = i
                    pt = (pt + 1) % BESAR
                    ditulis += 1
                kirim.release()
            else:
                s = pt
                pt = (pt + 1) % BESAR
                buf[s] = i
                ditulis += 1

    ter = set()

    def kons():
        nonlocal pb, dibaca, duplikat, hilang
        for _ in range(N):
            if pakai_mutex:
                kirim.acquire()
                with kunci:
                    d = buf[pb]
                    buf[pb] = None
                    pb = (pb + 1) % BESAR
                    dibaca += 1
                kosong.release()
            else:
                s = pb
                pb = (pb + 1) % BESAR
                d = buf[s]
                if d is None:
                    hilang += 1
                    continue
                dibaca += 1
            if d in ter:
                duplikat += 1
            else:
                ter.add(d)

    a = threading.Thread(target=prod, name="produsen")
    b = threading.Thread(target=kons, name="konsumen")
    a.start()
    b.start()
    a.join()
    b.join()
    return {"ditulis": ditulis, "dibaca": dibaca, "hilang": hilang,
            "duplikat": duplikat, "unik": len(ter)}


if __name__ == "__main__":
    ulang = int(sys.argv[1]) if len(sys.argv) > 1 else 5
    hasil = {"tanpa_mutex": [], "dengan_mutex": []}
    for _ in range(ulang):
        hasil["tanpa_mutex"].append(jalankan(False))
        hasil["dengan_mutex"].append(jalankan(True))

    for k, v in hasil.items():
        print(f"=== {k} ===")
        for h in v:
            print(json.dumps(h))
        unik_rata = sum(x["unik"] for x in v) / len(v)
        print(f"rata-rata unik terkirim: {unik_rata:.1f} dari {N} "
              f"({unik_rata / N * 100:.1f}%)")

    os.makedirs("results", exist_ok=True)
    with open("results/uji-tanpa-jeda.json", "w", encoding="utf-8") as f:
        json.dump({"target": N, "ulang": ulang, "hasil": hasil}, f, indent=2)
    print("tersimpan: results/uji-tanpa-jeda.json")
```

A.figures.py

```
"""Grafik untuk laporan I-4: kehilangan data dan duplikasi sebelum vs sesudah."""
import json
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

FONT = "DejaVu Sans"
OUT = "results"
BIRU, ORANYE, HIJAU, MERAH = "#3B5B8C", "#B7791F", "#2F6B3C", "#A03B2C"


def baca(nama):
    with open(f"results/{nama}.json", encoding="utf-8") as f:
        return json.load(f)


def grafik_unik(tanpa, dengan, uji, path):
    """Persentase data unik yang benar-benar sampai ke pengirim."""
    target = tanpa["target"]

    def persen(daftar):
        return [h["unik"] / target * 100 for h in daftar]

    fig, ax = plt.subplots(figsize=(7.4, 3.3))
    kiri = persen(tanpa["hasil"])
    kanan = persen(dengan["hasil"])
    x1 = list(range(len(kiri)))
    x2 = list(range(len(kanan), len(kiri) + len(kanan)))
    ax.set_xticks(x1 + x2)
    ax.set_xticklabels([str(i + 1) for i in range(len(kiri) + len(kanan))])

    ax.bar(x1, kiri, color=MERAH, edgecolor="white", width=0.7,
           label="tanpa mutex")
    ax.bar(x2, kanan, color=HIJAU, edgecolor="white", width=0.7,
           label="dengan mutex")
    ax.axhline(100, color="#888888", linewidth=0.8, linestyle="--")
    ax.text(len(kiri) / 2, 101, "target 100%", ha="center", fontsize=7,
            fontfamily=FONT, color="#666666")

    ax.set_ylim(0, 112)
    ax.set_ylabel("Data unik terkirim (%)", fontfamily=FONT, fontsize=8.5)
    ax.set_xlabel("Percobaan ke-", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="y", color="#DDDDDD", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.legend(frameon=False, fontsize=8, loc="lower right")
    ax.set_title("Data unik yang sampai ke konsumen, sebelum dan sesudah perbaikan",
                 fontsize=9.6, fontfamily=FONT, fontweight="bold", pad=8)
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def grafik_galat(tanpa, path):
    """Jumlah data hilang dan duplikat per percobaan."""
    fig, ax = plt.subplots(figsize=(7.4, 3.1))
    x = range(len(tanpa["hasil"]))
    hilang = [h["hilang"] for h in tanpa["hasil"]]
    duplikat = [h["duplikat"] for h in tanpa["hasil"]]

    ax.bar([i - 0.19 for i in x], duplikat, width=0.36, color=ORANYE,
           edgecolor="white", label="duplikat")
    ax.bar([i + 0.19 for i in x], hilang, width=0.36, color=BIRU,
           edgecolor="white", label="hilang")
    for i, (d, h) in enumerate(zip(duplikat, hilang)):
        ax.annotate(str(d), (i - 0.19, d), textcoords="offset points",
                    xytext=(0, 2), ha="center", fontsize=6.6, fontfamily=FONT)
        ax.annotate(str(h), (i + 0.19, h), textcoords="offset points",
                    xytext=(0, 2), ha="center", fontsize=6.6, fontfamily=FONT)

    ax.set_xticks(list(x))
    ax.set_xticklabels([str(h["run"]) for h in tanpa["hasil"]])
    ax.set_xlabel("Percobaan ke-", fontfamily=FONT, fontsize=8.5)
    ax.set_ylabel("Jumlah rekaman", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="y", color="#DDDDDD", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.set_ylim(0, max(duplikat) * 1.22)
```

```
    ax.legend(frameon=False, fontsize=8)
    ax.set_title("Rekaman duplikat dan hilang pada versi tanpa mutex",
                 fontsize=9.6, fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def grafik_skala(uji, path):
    """Uji tanpa jeda: memperlihatkan race bukan efek sleep."""
    target = uji["target"]
    tanpa = [h["unik"] / target * 100 for h in uji["hasil"]["tanpa_mutex"]]
    dengan = [h["unik"] / target * 100 for h in uji["hasil"]["dengan_mutex"]]

    fig, ax = plt.subplots(figsize=(6.4, 3.0))
    posisi = [0, 1]
    nilai = [sum(tanpa) / len(tanpa), sum(dengan) / len(dengan)]
    batang = ax.bar(posisi, nilai, color=[MERAH, HIJAU], width=0.5,
                    edgecolor="white")
    for b, v in zip(batang, nilai):
        ax.annotate(f"{v:.1f}%", (b.get_x() + b.get_width() / 2, v),
                    textcoords="offset points", xytext=(0, 4), ha="center",
                    fontsize=9, fontfamily=FONT, fontweight="bold")

    ax.set_xticks(posisi)
    ax.set_xticklabels(["tanpa mutex", "dengan mutex"], fontfamily=FONT,
                       fontsize=9)
    ax.set_ylim(0, 115)
    ax.set_ylabel("Data unik terkirim (%)", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="y", color="#DDDDDD", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.set_title(f"Uji tanpa jeda buatan ({uji['target']} rekaman per thread)",
                 fontsize=9.6, fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def main():
    os.makedirs(OUT, exist_ok=True)
    tanpa = baca("tanpa-mutex")
    dengan = baca("dengan-mutex")
    uji = baca("uji-tanpa-jeda")
    grafik_unik(tanpa, dengan, uji, f"{OUT}/gambar-perbandingan.png")
    grafik_galat(tanpa, f"{OUT}/gambar-kehilangan.png")
    grafik_skala(uji, f"{OUT}/gambar-tanpa-jeda.png")
    print("gambar:", [f for f in sorted(os.listdir(OUT)) if f.endswith(".png")])


if __name__ == "__main__":
    main()
```

A.ambil-bukti.sh

```
#!/usr/bin/env bash
# Bukti terminal asli untuk laporan I-4: race condition dan perbaikannya.
set -u

DIR=${1:?butuh folder kerja}
OUT=${2:?butuh folder keluaran}
FIFO=/tmp/shotest/i4.fifo
mkdir -p "$OUT"
rm -f "$FIFO"; mkfifo "$FIFO"

cd "$DIR" || exit 1

io.elementary.terminal -n -w "$DIR" -x "bash --noprofile --rcfile $HOME/.bashrc -i < $FIFO" \
  > /dev/null 2>&1 &
TERM_PID=$!
sleep 4

exec 3>"$FIFO"

tangkap() {
  sleep 1.0
  gdbus call --session --dest org.gnome.Shell.Screenshot \
    --object-path /org/gnome/Shell/Screenshot \
    --method org.gnome.Shell.Screenshot.ScreenshotWindow \
    true false false "$OUT/ss-$1.png" > /dev/null 2>&1
  echo "  -> ss-$1.png"
}
```

```
ketik() { printf '%s\n' "$1" >&3; }

ketik 'clear'
sleep 1
tangkap "01-prompt"

ketik 'grep -n "critical section" -A3 -B3 race_tanpa_mutex.py | head -14'
tangkap "02-critical-section"

ketik 'python3 race_tanpa_mutex.py --ulang 5'
sleep 4
tangkap "03-tanpa-mutex"

ketik 'python3 race_dengan_mutex.py --ulang 5'
sleep 4
tangkap "04-dengan-mutex"

ketik 'python3 uji_tanpa_jeda.py 3'
sleep 6
tangkap "05-tanpa-jeda"

ketik 'grep -n "kunci\|slot_kosong\|slot_terisi" race_dengan_mutex.py | head -8'
tangkap "06-mutex-semafor"

exec 3>&-
sleep 1
kill "$TERM_PID" 2>/dev/null
rm -f "$FIFO"
echo "selesai: $(ls -1 "$OUT"/ss-*.png | wc -l) tangkapan layar"
```

Lampiran B. Pernyataan Penggunaan AI Generatif

☐ Saya tidak menggunakan AI generatif dalam tugas ini.

☒ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude (Anthropic)	Menyusun skrip pengujian dan merapikan kalimat laporan	Bagian 4 (kode), Bagian 5.2	Seluruh angka berasal dari keluaran program pada tangkapan layar; kedua versi dijalankan ulang sepuluh kali dan hasilnya dicocokkan dengan Tabel 5.1

Lampiran C. Daftar Periksa Sebelum Mengumpulkan

☒ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.
☒ Identitas, kode tugas, dan nama host pada sampul sudah benar.
☒ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.
☒ Setiap tangkapan layar menampilkan prompt dan memiliki nomor serta keterangan.
☒ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.
☒ Pernyataan penggunaan AI generatif sudah diisi.
☒ Berkas disimpan sebagai PDF dengan nama C_I-4_<NIM>_<Nama>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana
Ketepatan konsep	35%	Ya — critical section, mutex, dan semafor dijelaskan dengan perannya masing-masing

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana
Kedalaman analisis	30%	Ya — mengapa duplikat mendominasi, mengapa perlu dua alat, dan biaya perbaikan
Bukti praktik/perhitungan	25%	Ya — 6 tangkapan layar asli, 10 percobaan per versi, uji tanpa jeda
Orisinalitas dan kerapian	10%	Ya — grafik dari data uji, daftar pustaka gaya APA